Firebase Google Cloud React 19 LangGraph + AI Paddle

# Pocket — System Design Document

Read-later web app with AI summaries, text-to-speech, and reading streaks

Version 1.3 | March 2026 | Project ID: `finn-2c4c5` | Region: `us-central1`

---

CONTENTS

# 1. System Overview

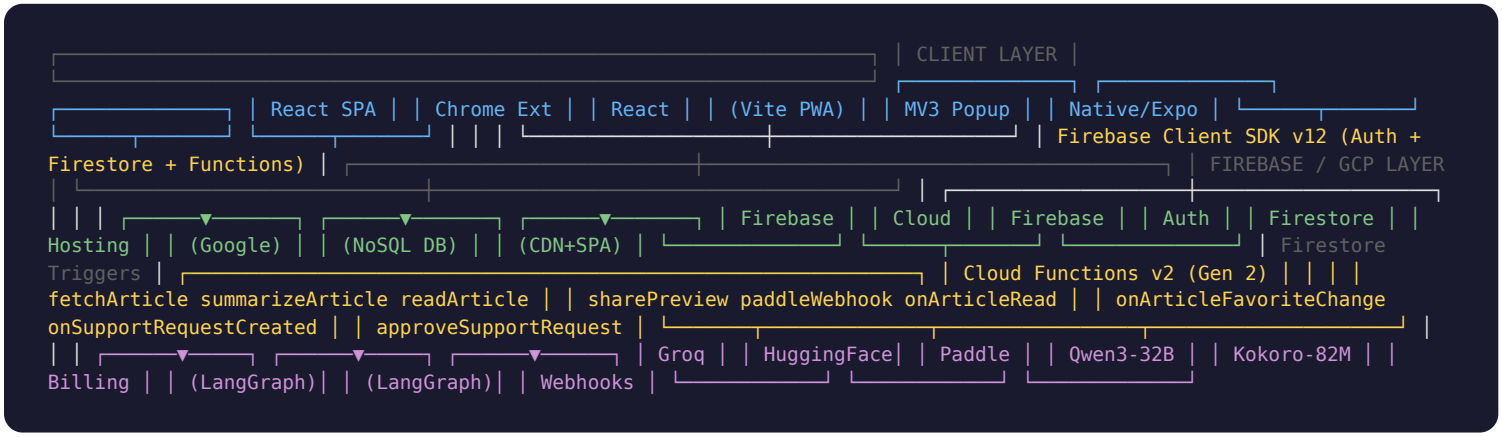
Pocket is a full-stack read-later application built on the Firebase ecosystem. Users save articles from the web, which are server-side fetched, parsed, and stored in Firestore. Premium features include AI-powered summaries (Groq LLM via LangGraph), AI text-to-speech (HuggingFace Kokoro), highlights, collections, and reading streak gamification.

| 9 | 5 | 15 | 2 |
|---|---|---|---|
| CLOUD FUNCTIONS | FIRESTORE COLLECTIONS | CLIENT ROUTES | LANGGRAPH PIPELINES |

**TECH STACK**

⚛️ React 19 · ⚡ Vite 7 · 🔥 Firebase v12 · ☁️ Cloud Functions v2 · 🤖 LangGraph · 🧠 Groq (Qwen3-32B) · 🔊 Kokoro TTS · 💳 Paddle · 🧩 Chrome MV3

## 2. High-Level Architecture



# 3. Frontend Architecture

## 3.1 Project Structure

```
src/
├── App.jsx                # Route definitions + auth guards
├── main.jsx               # Vite entry point
├── index.css              # Global CSS variables + reset
│
├── components/            # Reusable UI components
│   ├── Layout.jsx         # Sidebar + header + mobile nav shell
│   ├── ArticleCard.jsx    # Article list item (used in MyList, Archive, etc.)
│   ├── AddArticleModal.jsx # URL input modal for saving articles
│   ├── ImportModal.jsx    # CSV batch import modal
│   ├── EditorPicks.jsx    # Curated article recommendations
│   ├── FeaturedArticle.jsx # Most-liked article (from articleStats)
│   ├── StreakBadge.jsx    # Reading streak flame + badges dropdown
│   ├── ShareModal.jsx     # Share link generation (Bitly-style)
│   ├── HighlightToolbar.jsx# Text selection highlighting (premium)
│   └── VideoPlayer.jsx    # YouTube embed player
│
├── pages/                 # Route-level components
│   ├── Landing.jsx        # Marketing page (guests only)
│   ├── Login.jsx          # Google OAuth sign-in
│   ├── MyList.jsx         # Main article list (home)
│   ├── Archive.jsx        # Archived articles
│   ├── Favorites.jsx      # Favorited articles
│   ├── Tags.jsx           # Tag filtering view
│   ├── Collections.jsx    # Premium: curated collections
│   ├── Reader.jsx         # Full article reader + TTS + summary
│   ├── SaveHandler.jsx    # /save?url=... share target handler
│   ├── Premium.jsx        # Pricing + checkout
│   ├── Support.jsx        # Support ticket form
│   ├── About.jsx          # Version history + release notes
│   └── BlogReadingToolkit.jsx # Internal blog post
│
├── context/
│   ├── AuthContext.jsx     # Firebase Auth state + user profile
│   └── ThemeContext.jsx    # Light/dark theme toggle
│
├── firebase/
│   ├── config.js           # Firebase app initialization
│   ├── auth.js             # Auth helpers (login, logout, onAuthChange)
│   └── articles.js         # Firestore CRUD for articles + user profiles
│
├── hooks/
│   ├── usePremium.js       # Premium status hook
│   └── useScrollRestore.js  # Scroll position restoration on navigation
│
└── utils/
    └── articleFetcher.js    # Cloud Function callable wrappers
```

## 3.2 Route Map

| PATH | COMPONENT | AUTH | LAYOUT |
|---|---|---|---|
| `/` | Landing (guest) / MyList (user) | Dynamic | Conditional |
| `/archive` | Archive | Required | Yes |

| PATH | COMPONENT | AUTH | LAYOUT |
|---|---|---|---|
| `/favorites` | Favorites | Required | Yes |
| `/tags` , `/tags/:tag` | Tags | Required | Yes |
| `/collections` | Collections | Required | Yes |
| `/read/:id` | Reader | Required | No (fullscreen) |
| `/save?url=...` | SaveHandler | Required | No |
| `/premium` | Premium | Required | Yes |
| `/support` | Support | Required | Yes |
| `/about` | About | Required | Yes |
| `/p/:code` | ShareRedirect | No | No |
| `/blog/reading-toolkit` | BlogReadingToolkit | No | No |
| `/terms` , `/privacy` , `/refund` | Legal pages | No | No |
| `/login` | Login | Guest only | No |

### 3.3 State Management

```
ThemeProvider ← localStorage ('theme': 'light' | 'dark') | AuthProvider ← Firebase Auth + Firestore user profile | ├── user: Firebase User object (uid, email, displayName) | ├── userProfile: Firestore /users/{uid} doc (isPremium, streaks, badges) | ├── isPremium: boolean (derived from userProfile) | └── refreshProfile(): manually refetch profile from Firestore | └── Component Tree ├── useAuth() → access user + profile + premium status ├── usePremium() → isPremium + refreshProfile shorthand └── useTheme() → theme + toggle()
```

### 3.4 Build Configuration

| SETTING | VALUE |
|---|---|
| Bundler | Vite 7 + React plugin |
| Output | `dist/` (SPA with `index.html` fallback) |
| Code splitting | Manual chunks: `firebase` , `readability` , `router` , `vendor` |
| Bundle size | ~328KB app + ~359KB firebase (gzipped: ~103KB + ~112KB) |
| Styling | CSS Modules ( `*.module.css` ) |
| PWA | Service worker ( `sw.js` ) + `manifest.webmanifest` |

## 4. Backend — Cloud Functions

All Cloud Functions are defined in `functions/index.js` (CommonJS, Node 20). They run on Cloud Functions v2 (Gen 2) in `us-central1`.

| FUNCTION | TRIGGER | MEMORY | TIMEOUT | SECRETS | PURPOSE |
|---|---|---|---|---|---|
| `fetchArticle` | onCall (HTTPS) | 512 MiB | 30s | — | Server-side article fetcher. JSDOM + Readability parsing with Wayback Machine fallback. Handles Twitter/oEmbed, YouTube transcripts. |
| `summarizeArticle` | onCall (HTTPS) | 512 MiB | 60s | GROQ_API_KEY | LangGraph pipeline: assess word count → LLM summarize (Groq Qwen3-32B). Returns summary, key points, suggested tags. |
| `readArticle` | onCall (HTTPS) | 512 MiB | 120s | HF_API_KEY | LangGraph pipeline: prepare text chunks → synthesize via Kokoro TTS (HuggingFace). Returns base64 audio chunks. |
| `sharePreview` | onRequest (HTTP) | 256 MiB | 15s | — | OG meta tag server for share links. `/p/{code}` → reads `/shares/{code}` → returns HTML with OG tags + redirect. |
| `paddleWebhook` | onRequest (HTTP) | 256 MiB | 15s | PADDLE_WEBHOOK_SECRET | Paddle Billing webhook. HMAC-SHA256 signature verification. Sets/revokes `isPremium` on user doc. |
| `onArticleFavoriteChange` | Firestore trigger | 256 MiB | — | — | Maintains `/articleStats/{urlHash}` favorite counts when `isFavorite` toggles. |
| `onArticleRead` | Firestore trigger | 256 MiB | — | — | Updates reading streak, total articles read, and awards badges when `isRead` flips to true. |
| `onSupportRequestCreated` | Firestore trigger | 256 MiB | — | GMAIL_APP_PASSWORD | Sends email notification to admin when a support ticket is created. |
| `approveSupportRequest` | onCall (HTTPS) | 256 MiB | — | GITHUB_TOKEN | Admin-only. Creates GitHub issue with `claude-` |

| FUNCTION | TRIGGER | MEMORY | TIMEOUT | SECRETS | PURPOSE |
|---|---|---|---|---|---|
| | | | | | `task` label from approved support requests. |

## 4.1 Article Fetch Pipeline

```
Client ──► fetchArticle Cloud Function │ ├── Is Twitter/X? ──► oEmbed API ──► sanitize HTML ──► return │ ├── Is YouTube? ──► extract ytInitialPlayerResponse │ ├── title, thumbnail, description │ └── fetch captions (JSON3) ──► parse transcript │ └── Regular URL │ ├── fetch(url, headers) ──► HTTP 200? ──► parse │ ├── HTTP 403/429/503? │ └── Wayback Machine fallback │ web.archive.org/web/20260101000000/{url} │ └── Parse HTML ├── JSDOM + Readability (primary) ├── Fallback selectors if Readability returns null: │ article, [role="main"], main, .post-content, │ .entry-content, .article-body, #content, etc. ├── OG meta extraction (title, image, description) └── sanitize-html (whitelist tags + safe CSS)
```

## 4.2 Article Fetch — Known Failure Modes

| FAILURE | CAUSE | MITIGATION |
|---|---|---|
| JavaScript-rendered SPA | JSDOM doesn't execute JS. React/Next.js apps return empty `<div id="root">` | OG meta returned; content empty. Future: Puppeteer headless browser |
| Paywall / login wall | HTTP 403 or JS-gated content | Wayback Machine fallback for archived snapshots |
| Bot blocking (Cloudflare, Akamai) | Cloud Function IPs flagged as bots | Custom User-Agent + Wayback fallback |
| Rate limiting | HTTP 429 from publisher | Wayback Machine fallback |
| Readability returns null | Non-standard HTML structure | 14 fallback CSS selectors tried in order |

# 5. Data Model — Firestore

```
Firestore Database │ ├── users/{uid} ← user profile + streak data │ │ uid, displayName, email, photoURL, createdAt │ │ isPremium, premiumSince, premiumEndedAt │ │ paddleCustomerId, paddleSubscriptionId │ │ currentStreak, longestStreak, lastReadDate │ │ totalArticlesRead, badges[] │ │ │ └── articles/{articleId} ← saved articles │ │ url, title, excerpt, heroImage, domain │ │ content, wordCount, estimatedReadTime, authors[] │ │ tags[], fetchStatus, fetchError │ │ isRead, isFavorite, isArchived │ │ readAt, savedAt │ │ aiSummary, aiKeyPoints[], aiSuggestedTags[] │ │ isVideo, videoId, transcript[] │ │ │ │ └── highlights/{highlightId} ← text highlights (premium) │ text, color, note, startOffset, endOffset, createdAt │ ├── shares/{code} ← Bitly-style share links │ url, title, heroImage, excerpt, domain, createdAt │ ├── articleStats/{urlHash} ← global favorite counts │ url, title, heroImage, excerpt, domain │ favoriteCount, lastUpdated │ ├── supportRequests/{requestId} ← support tickets │ userId, userName, userEmail, title, description │ status (pending/approved/rejected/done) │ createdAt, approvedAt, rejectedAt │ githubIssueUrl, githubIssueNumber │ └── errors/{autoId} ← function error logs type, uid, articleId, title, error, timestamp
```

## 5.1 Firestore Indexes

| COLLECTION | FIELDS | SCOPE |
|---|---|---|
| `supportRequests` | `userId` ASC + `createdAt` DESC | Collection |
| `articleStats` | `favoriteCount` DESC (single-field, auto) | Collection |

> **Design decision:** Articles are sorted client-side ( `savedAt` descending) to avoid composite index requirements. With a 500-article free-tier cap and unlimited for premium, this keeps Firestore costs minimal while avoiding index proliferation.

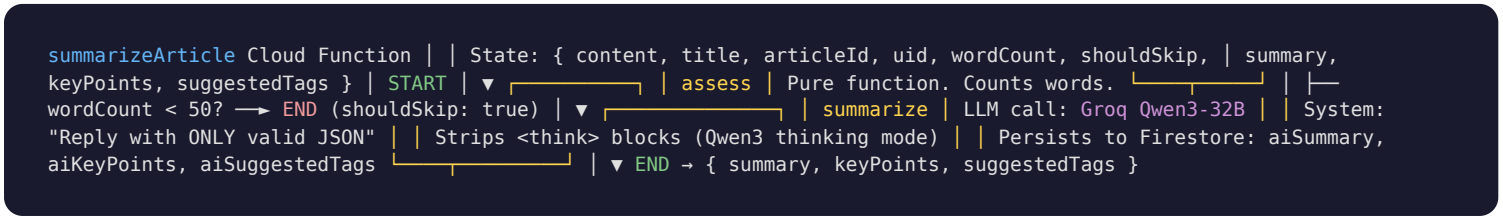
## 5.2 Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users: read/write own doc only. Cannot self-grant isPremium.
    match /users/{userId} {
      allow read:   if auth.uid == userId;
      allow create: if auth.uid == userId && !data.isPremium;
      allow update: if auth.uid == userId
                    && newData.isPremium == existingData.isPremium;

      // Articles + highlights: full CRUD for owner
      match /articles/{articleId} { allow read, write: if auth.uid == userId; }
      match /articles/{articleId}/highlights/{hId} { allow read, write: if auth.uid == userId; }
    }

    // Shares: anyone reads (crawlers), auth users create
    match /shares/{id} { allow read: if true; allow create: if auth != null; }

    // ArticleStats: read-only for clients (written by Cloud Function admin SDK)
    match /articleStats/{id} { allow read: if auth != null; }

    // Support: create own, read own + admin, admin can update status
    match /supportRequests/{id} {
      allow create: if auth != null && data.userId == auth.uid;
      allow read:   if data.userId == auth.uid || auth.email == ADMIN;
      allow update: if auth.email == ADMIN;
    }
  }
}
```

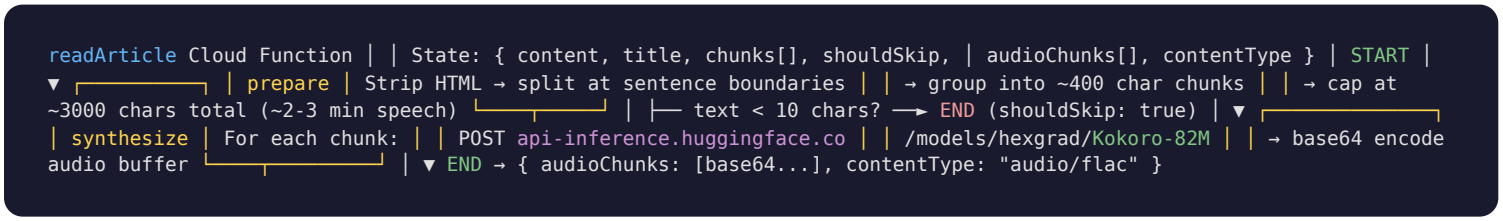
# 6. AI Pipeline — LangGraph

Two AI features are implemented as LangGraph StateGraph pipelines inside Cloud Functions. Each pipeline defines typed state, node functions, conditional routing, and compiles to an executable graph.

## 6.1 Summarization Pipeline




## 6.2 Text-to-Speech Pipeline




> **LangGraph pattern:** Both pipelines use `Annotation.Root()` for typed state, `addConditionalEdges()` for routing, and `compile()` for immutable graph execution. The state flows through each node, with partial updates merged automatically.

# 7. Authentication & Authorization

```
User ──► Firebase Auth (Google OAuth 2.0) │ ├── Sign in → onAuthChange listener fires │ ├── Sets user state (uid, email, displayName) │ └── Fetches /users/{uid} from Firestore │ └── Sets userProfile (isPremium, streaks, etc.) │ ├── ProtectedRoute guard │ If !user → store intended path in sessionStorage │ → redirect to /login │ If user → render children │ ├── PublicRoute guard │ If user → redirect to / │ If !user → render children (login page) │ └── Cloud Functions request.auth.uid → used for Firestore doc ownership request.auth.token.email → admin check (support approval)
```

**AUTHORIZATION LEVELS**

| ROLE | ACCESS |
|---|---|
| Guest | Landing page, login, blog, legal pages, share redirects |
| Free user | Save up to 500 articles, browser TTS, basic search, tags, PWA |
| Premium user | All free features + AI TTS, AI summaries, highlights, collections, unlimited saves |
| Admin | Approve/reject support requests → creates GitHub issues |

# 8. Payment System — Paddle

```
Premium Page │ ├── Click "Subscribe" → Paddle.Checkout.open() │ customData: { user_id: firebase_uid } │ ├── User completes payment on Paddle's hosted checkout │ └── Paddle sends webhook ──► paddleWebhook Cloud Function │ ├── Verify HMAC-SHA256 signature (PADDLE_WEBHOOK_SECRET) │ signed = "ts:rawBody" │ expected = HMAC(secret, signed) │ timing-safe comparison │ ├── Extract user_id from custom_data │ ├── subscription.activated / .created / transaction.completed │ └── Set isPremium: true, premiumSince, paddleCustomerId │ └── subscription.canceled / .paused └── Set isPremium: false, premiumEndedAt
```

**PRICING**

| PLAN | PRICE | NET (AFTER PADDLE FEES) |
|---|---|---|
| Free | $0 | — |
| Premium Monthly | $3.99/mo | ~$3.23/mo |
| Premium Annual | $29.99/yr ($2.50/mo) | ~$27.54/yr |

# 9. CI/CD & Deployment

```
git push to master or claude/* | ├── deploy.yml (always runs) | ├── npm ci + npm run build | ├── Deploy Firestore rules + indexes | └── Deploy to Firebase Hosting (dist/ → CDN) | └── deploy-functions.yml (only if functions/** changed) ├── npm ci (functions/) ├── Sync secrets to GCP Secret Manager: | GITHUB_TOKEN, GROQ_API_KEY, HF_API_KEY, | LEMON_SQUEEZY_API_KEY, LEMON_SQUEEZY_WEBHOOK_SECRET | └── Deploy each function individually via gcloud: ├── fetchArticle (gcloud functions deploy, Gen 2) ├── summarizeArticle (gcloud + GROQ_API_KEY secret) ├── readArticle (gcloud + HF_API_KEY secret) ├── sharePreview (gcloud, 256MiB) ├── paddleWebhook (gcloud + webhook secret) ├── onArticleFavoriteChange (firebase deploy, Eventarc trigger) ├── onArticleRead (firebase deploy, Eventarc trigger) ├── onSupportRequestCreated (firebase deploy + GMAIL_APP_PASSWORD) └── approveSupportRequest (gcloud + GITHUB_TOKEN secret)
```

> **Why gcloud instead of firebase deploy?** HTTP-triggered functions are deployed via `gcloud functions deploy` for precise control over memory, timeout, and secrets. Firestore-triggered functions use `firebase deploy` because Eventarc setup requires it.

#### GITHUB SECRETS

| SECRET | USED BY | PURPOSE |
|---|---|---|
| `FIREBASE_SERVICE_ACCOUNT` | Both workflows | GCP service account JSON for deployment |
| `GROQ_API_KEY` | deploy-functions | Groq LLM API for summarization |
| `HF_API_KEY` | deploy-functions | HuggingFace API for Kokoro TTS |
| `GHUB_PAT` | deploy-functions | GitHub PAT → GITHUB_TOKEN secret for issue creation |
| `LEMON_SQUEEZY_API_KEY` | deploy-functions | Legacy (migrated to Paddle) |
| `LEMON_SQUEEZY_WEBHOOK_SECRET` | deploy-functions | Legacy (migrated to Paddle) |

## 10. Chrome Extension

Manifest V3 Chrome extension that lets users save the current tab to their Pocket list.

```
extension/
├── manifest.json        # MV3 manifest (permissions: activeTab, storage, identity)
├── src/popup.js         # Source → built via vite.config.extension.js
├── popup.html           # Extension popup UI
├── popup.bundle.js      # Built IIFE bundle (vite build:ext)
└── icons/               # Extension icons (16, 48, 128)
```

```
User clicks extension icon │ ├── popup.html loads popup.bundle.js ├── Gets current tab URL via chrome.tabs.query() ├──
Authenticates via Firebase Auth (cached token in chrome.storage) ├── Calls addArticle() to save URL to Firestore ├──
Fires fetchArticle Cloud Function in background └── Shows "Saved!" confirmation
```

# 11. Security Architecture

**DEFENSE IN DEPTH**

| LAYER | MECHANISM |
| --- | --- |
| Authentication | Firebase Auth (Google OAuth 2.0). All Cloud Functions check `request.auth`. |
| Authorization | Firestore rules enforce doc ownership (`auth.uid == userId`). Premium flag can only be set by Cloud Function admin SDK. |
| Input sanitization | `sanitize-html` with whitelist of safe tags and CSS properties. XSS prevention on article content. |
| Webhook verification | HMAC-SHA256 signature verification for Paddle webhooks. `crypto.timingSafeEqual` prevents timing attacks. |
| Secret management | GCP Secret Manager via `defineSecret()`. Secrets synced from GitHub Secrets during CI/CD. |
| URL validation | Protocol whitelist (`http:`, `https:` only). Domain-specific handling for social URLs. |
| Content Security | CSS properties whitelisted (no `position:fixed`, `z-index`). Links forced to `target="_blank" rel="noopener noreferrer"`. |
| Admin access | Email-based admin check (`request.auth.token.email === ADMIN_EMAIL`) |

# 12. Scaling & Cost Analysis

## 12.1 Serverless Auto-Scaling

All infrastructure is serverless (Firebase Hosting CDN + Cloud Functions + Firestore). Zero fixed costs — scales to zero when idle, scales up automatically under load.

| ~$0.002 | ~200 DAU | ~100 DAU | #1 |
|---|---|---|---|
| COST PER USER/MONTH | GROQ FREE TIER LIMIT | HUGGINGFACE FREE TIER LIMIT | PROFITABLE FROM SUBSCRIBER |

## 12.2 Firestore Query Patterns

| QUERY | COLLECTION | COMPLEXITY | NOTES |
|---|---|---|---|
| Get user's articles (filtered) | `users/{uid}/articles` | O(n) scan, n ≤ 500 | Free tier capped at 500. Client-side sort avoids composite indexes. |
| Get featured article | `articleStats` | O(1) | Single query with `orderBy('favoriteCount', 'desc').limit(1)` |
| Check article exists by URL | `users/{uid}/articles` | O(1) | `where('url', '==', url).limit(1)` — single-field auto index |
| Count articles | `users/{uid}/articles` | O(1) | `getCountFromServer()` — doesn't download docs |
| User's support requests | `supportRequests` | O(n) | Composite index: `userId` ASC + `createdAt` DESC |

## 12.3 Scaling Bottlenecks

| COMPONENT | CURRENT LIMIT | WHEN IT BREAKS | SOLUTION |
|---|---|---|---|
| Groq free tier | ~30 req/min | ~200 DAU | Upgrade to Groq paid tier ($0.05/M input tokens) |
| HuggingFace free tier | ~100 req/hour | ~50-100 DAU | Self-host Kokoro or upgrade HF Pro ($9/mo) |
| Firestore reads | 50K/day (free tier) | ~500 DAU | Blaze plan ($0.06/100K reads) |
| Cloud Function invocations | 2M/mo (free tier) | ~2000 DAU | Already on Blaze plan for external HTTP |
| Client-side sorting | 500 docs max | Premium heavy users | Server-side pagination + composite indexes |

## 13. Monitoring & Observability

| SIGNAL | TOOL | LOCATION |
|---|---|---|
| Function logs | Cloud Logging ( `logger.info/error` ) | Firebase Console → Functions → Logs |
| Error tracking | Firestore `/errors` collection | Written by summarizeArticle on failure |
| Analytics | Firebase Analytics ( `G-039HM57SZ4` ) | GA4 dashboard |
| Uptime | Firebase Hosting CDN | 99.95% SLA |
| Deployment status | GitHub Actions | Two workflows: hosting + functions |
| Support tickets | Email (Gmail) + Firestore | onSupportRequestCreated → admin email notification |
| Payment events | Paddle Dashboard + Cloud Logging | paddleWebhook logs all events |

> **Recommended alert:** Set up a Cloud Logging alert on `severity=ERROR` for the `summarizeArticle` and `readArticle` functions to get notified when AI API calls fail.